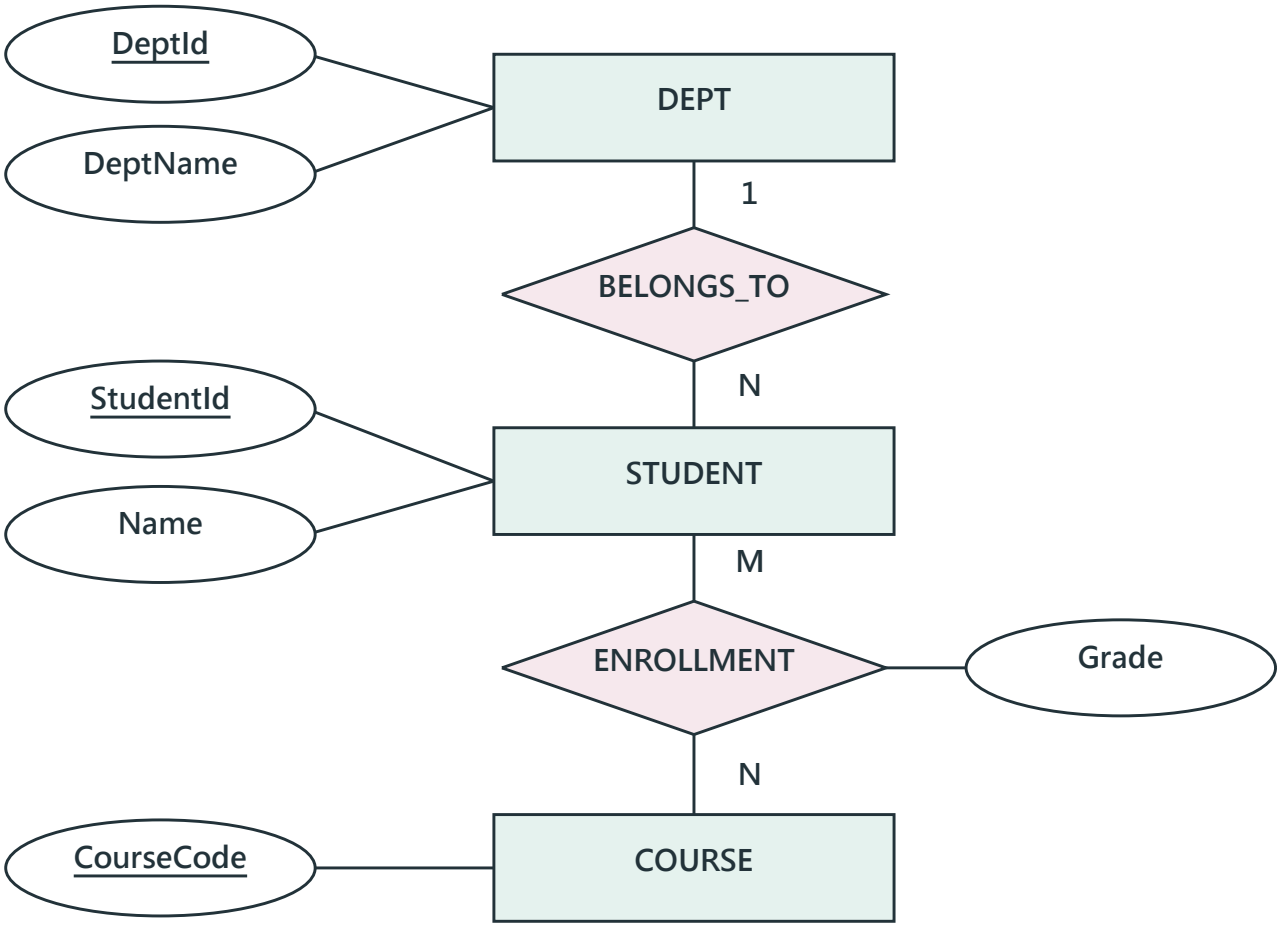
選課案例：學生、系所與課程

教務處記錄同一學期的系所、學生、課程與選課成績。本章先看完整關係與資料，再依課本順序建表、加限制、查詢、修改。所有人物與數值都是人工教學資料。



矩形是實體，菱形是關係，橢圓是屬性；底線標出各實體的識別欄位。Grade 接在選課關係上。

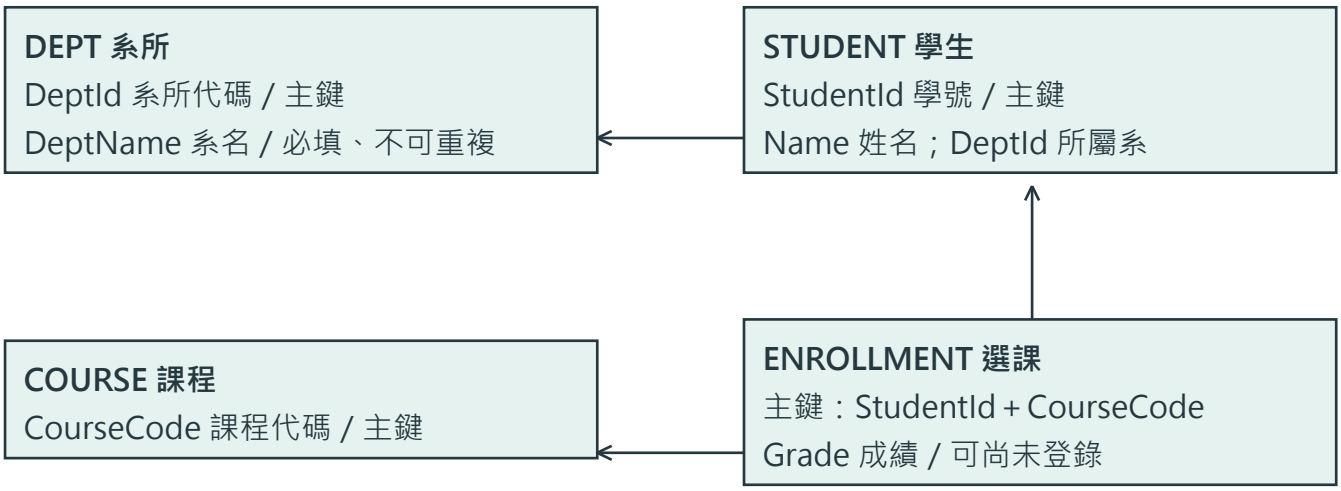
本校先訂好的規則

關係	可以怎麼連？
學生與系所	一位學生屬於 0 或 1 個系；一個系可有 0 到多位學生。
學生與課程	一位學生可修多門課；一門課可有多位學生，也都可以暫無配對。
每筆選課	同一學生同一課程只記一筆成績：0 至 100 的整數，或尚未登錄。

本頁重點：成績屬於「哪位學生修哪門課」，不能只接在學生或課程上。

選課案例：四張表的關聯

學生可未分系，也可尚未選課。每筆選課必須指向一位已有學生與一門已有課程。



箭頭：依所存代碼，到另一張表查相同代碼的列。

學生可尚未分系；選課必須指向已存在的學生與課程。

每張表如何指出是哪一列？

表	識別欄位	要查到哪張表？
DEPT	DeptId	不參照其他表
STUDENT	StudentId	DeptId 查 DEPT；可尚未分系
COURSE	CourseCode	不參照其他表
ENROLLMENT	StudentId + CourseCode	StudentId 查 STUDENT；CourseCode 查 COURSE

識別一列的欄位稱主鍵（PRIMARY KEY）；存另一張表的代碼、要求查得到對象的欄位稱外來鍵（FOREIGN KEY）。圖中箭頭是查找方向，不是新增資料的先後。

BELONGS_TO 存成 STUDENT.DeptId；DeptId 為 NULL 表示未分系。ENROLLMENT 是多對多且帶成績，另建表，以學號 + 課號為主鍵。

本頁重點：主鍵指出本表哪一列；外來鍵要求所存代碼能查到另一張表的對象。

學校資料：八位學生與兩個系所

這是本章學校案例的固定原資料。StudentId 是學號，Name 是姓名，DeptId 是所屬系所；NULL 表示未分系。學號不必連號，S4 留給後面的新增練習。

StudentId	Name	DeptId
S1	Alex	D1
S2	Alex	D1
S3	Casey	NULL
S5	Morgan	D1
S6	Riley	D2
S7	Taylor	D2
S8	Jordan	D2
S9	Robin	NULL

DeptId	DeptName
D1	資訊管理
D2	會計

問題：兩位 Alex 是同一位學生嗎？

回答：不是。S1、S2 學號不同，是兩位學生。

本頁重點：D1 有 S1、S2、S5；D2 有 S6、S7、S8；S3、S9 未分系。

學校資料：八筆選課與三門課程

延續前頁八位學生。每筆選課用 StudentId + CourseCode 識別；Grade 是0到100的整數，NULL 表示成績尚未登錄。

顯示編號	StudentId	CourseCode	Grade
E1	S1	DB101	80
E2	S1	CS102	NULL
E3	S2	DB101	0
E4	S6	CS102	90
E5	S7	DB101	60
E6	S8	DB101	75
E7	S9	CS102	NULL
E8	S6	NW103	95

E1–E8 只是本教材標示這八列的編號，不是資料庫欄位，也不是主鍵。後面的配對矩陣用它們指出原列。

CourseCode
DB101
CS102
NW103

問題：S2 的0分與 S9 的 NULL 一樣嗎？

回答：不一樣。S2 已登錄0分；S9 尚未登錄。S3、S5 沒有選課紀錄，不是成績 NULL。

本頁重點：兩個 Alex 用學號區分；未分系、未選課、尚未登錄成績是三件不同的事。

SQL 操作：建表、載入與重設

先讀下一組建表程式；第一次建立的四張表仍空。查詢練習則一律使用剛才的八人、八筆選課，不沿用其他頁改過的資料。



這是執行順序，不是外來鍵參照方向。載入步驟在 INSERT 小節完整列出。

SELECT 只讀資料，可以連續執行。A / B 若改資料，會明說另開相同起點或接續哪一步；不要把 CREATE TABLE 重複貼進已有同名表的資料庫。

程式左側色條標出該頁要比較的條件或輸入。

實作額外補充：SQLite
PRAGMA foreign_keys = ON：開啟本連線的外來鍵檢查。
PRAGMA foreign_keys：查看是否為1；不是宣告哪一欄是外來鍵。

本頁重點：重設：開新資料庫→建表→載入。每次新連線都先執行 PRAGMA foreign_keys = ON，才會檢查外來鍵。

CREATE TABLE：系所表與課程表

第1步：新的無表 SQLite 資料庫。先建立不參照其他表的 DEPT、COURSE；預測執行後是否已有 D1。

```
PRAGMA foreign_keys = ON;
CREATE TABLE DEPT (
  DeptId TEXT NOT NULL PRIMARY KEY,
  DeptName TEXT NOT NULL UNIQUE
);
CREATE TABLE COURSE (
  CourseCode TEXT NOT NULL PRIMARY KEY
);
PRAGMA foreign_keys;
```

實際結果
DEPT、COURSE 都存在，但各0列。
最後的 PRAGMA 回傳1：此連線已開外來鍵檢查。

問題：PRIMARY KEY 與 UNIQUE 分別限制什麼？

回答：兩者都禁止重複；主鍵識別本表的列，一表只有一組。系名另用 UNIQUE 限制不重複。

本章 SQLite 普通表的 TEXT 主鍵明寫 NOT NULL，避免允許缺值的歷史例外；NOT NULL 要求必填。

本頁重點：CREATE TABLE 定義欄位與限制；不會自動加入圖上的資料。

欄位型別：代碼、姓名與成績

回看原資料：S1 是代碼，Alex 是姓名，80 是成績。先指定要保存的值，再選型別；成績範圍另用 CHECK 限制。

欄位	原資料例子	本例 SQLite 宣告
StudentId	S1	TEXT
Name	Alex	TEXT
Grade	80 或 NULL	INTEGER

課本介紹 CHAR、VARCHAR、INTEGER、DATE 等型別。本章可執行補充使用 SQLite：日期存固定年月日文字；不能直接把其他資料庫的型別行為套進來。

問題：Grade INTEGER 是否已保證只能存整數？

回答：在 SQLite 普通表中不保證。下一頁用同一個80.5，比較只有宣告與多加整數檢查。

本頁重點：資料型別與資料限制分開看；下面的執行結果以 SQLite 為準。

SQLite：拒絕小數成績的兩段程式

實作額外補充。相同目標：拒絕80.5，但允許整數與 NULL。A、B 各用新的無表資料庫；只有 B 多整數檢查。

A：需求是拒絕小數。INSERT 加資料，SELECT 查保存的值；typeof 讀儲存型別，real 表示實數。

```
CREATE TABLE GRADE_TEST (Grade INTEGER);
INSERT INTO GRADE_TEST VALUES (80.5);
SELECT Grade,typeof(Grade) FROM GRADE_TEST;
```

Grade	typeof(Grade)
80.5	real

A 存入成功，與目標不同；real 表示實際保存為實數。

B：typeof 讀儲存型別；'integer' 表示整數，CHECK 要求條件成立。

```
CREATE TABLE GRADE_TEST (
  Grade INTEGER CHECK (Grade IS NULL OR typeof(Grade)='integer')
);
INSERT INTO GRADE_TEST VALUES (80.5);
```

B 的 INSERT 報錯後，另執行與 A 相同的 SELECT，不重做建表。

```
SELECT Grade,typeof(Grade) FROM GRADE_TEST;
```

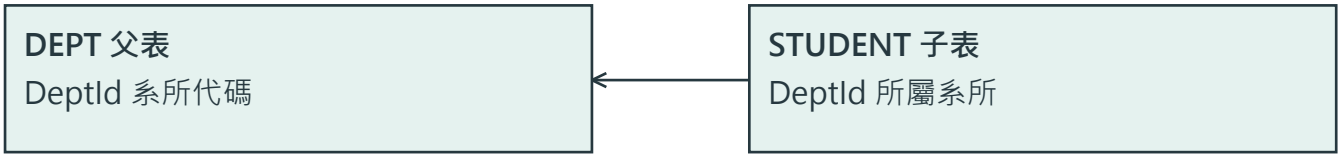
B：CHECK constraint failed
80.5 被拒絕；SELECT 回傳0列，測試表仍空。

本頁重點：答案：A 接受80.5；B 拒絕80.5。整數宣告不是這個 SQLite 測試的整數限制。

CREATE TABLE：學生表的欄位與外來鍵

第2步：接續第6頁。被參照的 DEPT 稱父表；存外來鍵的 STUDENT 稱子表。姓名必填但可同名；DeptId 可缺，省略時預設 D1。

```
CREATE TABLE STUDENT (
  StudentId TEXT NOT NULL PRIMARY KEY,
  Name TEXT NOT NULL,
  DeptId TEXT DEFAULT 'D1',
  FOREIGN KEY (DeptId) REFERENCES DEPT(DeptId)
    ON DELETE RESTRICT ON UPDATE CASCADE
);
```



改系所代碼，同步學生的 DeptId：ON UPDATE CASCADE

刪仍有學生參照的系所，拒絕刪除：ON DELETE RESTRICT

箭頭是 STUDENT.DeptId 參照 DEPT.DeptId。兩項設定都寫在子表 STUDENT 的建表程式。

問題：哪一行指定外來鍵欄位？

回答：FOREIGN KEY (DeptId) 指定學生表的 DeptId；REFERENCES DEPT(DeptId) 指向系所表的主鍵。

本頁重點：外來鍵寫在學生表；更改父表代碼與刪除父表列，分別由 UPDATE、DELETE 設定處理。

CREATE TABLE：選課的主鍵與成績限制

第3步：接續前頁，已有空 DEPT、COURSE、STUDENT。每人每課只記一筆；成績可為 NULL，否則要是0至100的整數。

```
CREATE TABLE ENROLLMENT (
  StudentId TEXT NOT NULL REFERENCES STUDENT(StudentId),
  CourseCode TEXT NOT NULL REFERENCES COURSE(CourseCode),
  Grade INTEGER,
  PRIMARY KEY (StudentId, CourseCode),
  CONSTRAINT grade_domain CHECK (Grade IS NULL OR
    (typeof(Grade) = 'integer' AND Grade BETWEEN 0 AND 100))
);
```

規則	對應原資料
兩欄共同主鍵	S1 / DB101 與 S1 / CS102 可並存
兩條 REFERENCES	學號與課號分別查學生、課程
CHECK	80、0 可存；80.5、101 不符合

欄位後直接寫 REFERENCES，是單欄外來鍵的寫法。

CONSTRAINT grade_domain 只是這項成績限制的名稱。typeof 是 SQLite 額外補充；CHECK 與 BETWEEN 的條件在本頁先讀成「缺值，或整數且介於0到100」。

四表此時仍空。先到第43、44頁載入資料，才有第3、4頁的內容。第11頁另開測試庫；第15頁再回完整原資料。

本頁重點：PRIMARY KEY 限制哪一筆選課；FOREIGN KEY 限制參照對象；CHECK 限制成績值。

選課主鍵 A：只用學號會拒絕第二門課

起點：第3、4頁的完整學校資料，八位學生、八筆選課。先另開測試庫：依第6、9、10頁建表，再依第43、44頁載入。INSERT 在6.4解說；此處先照貼，四表不改。

需求：S1 的兩門課都加入 KEY_TEST。A 只用 StudentId 作主鍵，先預測第二句 INSERT 是否成功。

```
CREATE TABLE KEY_TEST (
  StudentId TEXT NOT NULL REFERENCES STUDENT(StudentId),
  CourseCode TEXT NOT NULL REFERENCES COURSE(CourseCode),
  Grade INTEGER, PRIMARY KEY (StudentId)
);
INSERT INTO KEY_TEST VALUES ('S1','DB101',80);
INSERT INTO KEY_TEST VALUES ('S1','CS102',NULL);
```

第二句 INSERT 失敗後，另外執行 SELECT * 查看測試表的全部欄位；不重做建表或第一筆新增。

```
SELECT * FROM KEY_TEST;
```

StudentId	CourseCode	Grade
S1	DB101	80

第二句失敗：相同學號已存在，只留下第一門課。

本頁重點：只用學號限制成「每位學生最多一筆」，不是學校要的「每人每課一筆」。

選課主鍵 B：學號加課號可存兩門課

起點：第3、4頁的完整學校資料，八位學生、八筆選課。B 另用相同原資料，尚無 KEY_TEST；只把主鍵改成 StudentId、CourseCode，其餘欄位與兩筆輸入完全相同。

```
CREATE TABLE KEY_TEST (
  StudentId TEXT NOT NULL REFERENCES STUDENT(StudentId),
  CourseCode TEXT NOT NULL REFERENCES COURSE(CourseCode),
  Grade INTEGER, PRIMARY KEY (StudentId,CourseCode)
);
INSERT INTO KEY_TEST VALUES ('S1','DB101',80);
INSERT INTO KEY_TEST VALUES ('S1','CS102',NULL);
SELECT * FROM KEY_TEST;
```

StudentId	CourseCode	Grade
S1	DB101	80
S1	CS102	NULL

兩句 INSERT 都成功；同一學號搭配不同課號，是不同選課。

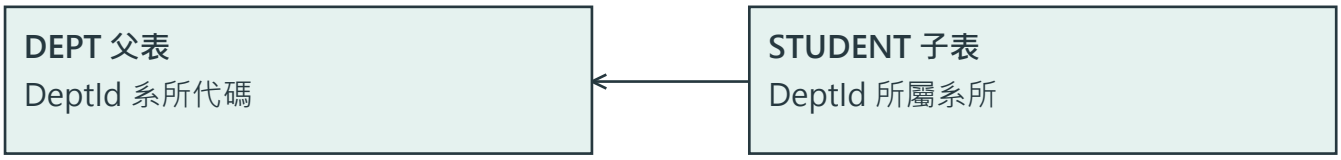
問題：再加入 S1 / DB101 的另一個分數會成功嗎？

回答：不會。這次兩欄組合重複；需要更正原列，不是新增另一列。

本頁重點：B 才符合每人每課一筆；姓名與成績不參與這筆選課的識別。

ON DELETE RESTRICT：保留被參照的系所

起點：第3、4頁的完整學校資料，八位學生、八筆選課。D1 被 S1、S2、S5 參照。學校要保留學生與成績，正式定義用 ON DELETE RESTRICT。DELETE 的完整語法在6.4再練。



改系所代碼，同步學生的 DeptId：ON UPDATE CASCADE

刪仍有學生參照的系所，拒絕刪除：ON DELETE RESTRICT

箭頭是 STUDENT.DeptId 參照 DEPT.DeptId。兩項設定都寫在子表 STUDENT 的建表程式。

預測刪 D1 是否成功；只執行下方一句。

```
DELETE FROM DEPT WHERE DeptId='D1';
```

實際：刪除被拒絕
DEPT：2列；D1、D2 都保留。
STUDENT：8列；ENROLLMENT：8列；COURSE：3列。

問題：刪 D1 被拒，是因為學生有選課嗎？

回答：不是。S1、S2、S5 的 DeptId 參照 D1；系所不能直接刪，整句不生效。

本頁重點：RESTRICT 拒絕刪除仍被學生參照的 D1；ON UPDATE CASCADE 不處理這句 DELETE。

ON DELETE SET NULL：保留學生並取消分系

另測一項需求：允許刪系所，但留下學生與成績。另開相同完整起點，只在第9頁把 ON DELETE RESTRICT 換成 ON DELETE SET NULL，其餘表與資料不變。



設定仍寫在子表 STUDENT，不是改 DELETE 語法。

```
DELETE FROM DEPT WHERE DeptId='D1';
SELECT StudentId,DeptId FROM STUDENT;
```

StudentId	DeptId
S1	NULL
S2	NULL
S3	NULL
S5	NULL
S6	D2
S7	D2
S8	D2
S9	NULL

本頁重點：S1、S2、S5 改為未分系；沒有刪除任何學生或選課。下一組查詢回到正式 RESTRICT 的原資料。

SELECT：查詢 D1 的學生姓名

起點：第3、4頁的完整學校資料，八位學生、八筆選課。不要接續前頁刪 D1 的測試。另開新資料庫，依第6、9、10頁建表，第43、44頁載入。SELECT 選姓名欄，WHERE 只留 D1 學生。

先預測結果，再執行。

```
SELECT Name FROM STUDENT
WHERE DeptId='D1';
```

Name
Alex
Alex
Morgan

圖按學號整理方便對照；SQL 未寫 ORDER BY，不保證列序。

本頁重點：S1、S2、S5 屬 D1，所以輸出 Alex、Alex、Morgan；SELECT 選欄，WHERE 篩列。

WHERE：兩門課都要檢查及格條件

起點：第3、4頁的完整學校資料，八位學生、八筆選課。目標是 DB101 或 CS102 中成績至少60的選課；NW103 即使95分也不在本次名單。

A 缺少括號，預測0分的 S2 是否也被留下。

```
SELECT StudentId,CourseCode,Grade FROM ENROLLMENT
WHERE CourseCode='DB101' OR CourseCode='CS102' AND Grade>=60;
```

B 只加括號，讓及格條件套在兩門課上。

```
SELECT StudentId,CourseCode,Grade FROM ENROLLMENT
WHERE (CourseCode='DB101' OR CourseCode='CS102') AND Grade>=60;
```

A：錯誤5列

StudentId	CourseCode	Grade
S1	DB101	80
S2	DB101	0
S6	CS102	90
S7	DB101	60
S8	DB101	75

B：正確4列

StudentId	CourseCode	Grade
S1	DB101	80
S6	CS102	90
S7	DB101	60
S8	DB101	75

AND 先於 OR。A 連 S2 的0分也留下；B 不留0分、NULL 或 NW103。NULL>=60 無法判定，見第24頁。

本頁重點：只有括號改變；A 多了 S2 / DB101 / 0，B 才符合相同需求。

兩表查詢：把姓名接到每筆選課

起點：第3、4頁的完整學校資料，八位學生、八筆選課。ENROLLMENT 沒存姓名，要用學號到 STUDENT 查姓名；這種查詢稱接合（join）。

先預測結果，再執行。

```
SELECT STUDENT.StudentId,STUDENT.Name,ENROLLMENT.CourseCode
FROM STUDENT,ENROLLMENT
WHERE STUDENT.StudentId=ENROLLMENT.StudentId;
```

StudentId	Name	CourseCode
S1	Alex	DB101
S1	Alex	CS102
S2	Alex	DB101
S6	Riley	CS102
S6	Riley	NW103
S7	Taylor	DB101
S8	Jordan	DB101
S9	Robin	CS102

圖按學號整理方便對照；SQL 未寫 ORDER BY，不保證列序。

本頁重點：八筆選課各查到一位學生；S1、S6 各有兩門課，所以各出現兩列。

AS：表別名與輸出欄名

起點：第3、4頁的完整學校資料，八位學生、八筆選課。a 是本句 STUDENT 的短名；StudentCode 是輸出學號欄的名稱。預測是否少一位學生。

先預測結果，再執行。

```
SELECT a.StudentId AS StudentCode,a.Name
FROM STUDENT AS a;
```

StudentCode	Name
S1	Alex
S2	Alex
S3	Casey
S5	Morgan
S6	Riley
S7	Taylor
S8	Jordan
S9	Robin

圖按學號整理方便對照；SQL 未寫 ORDER BY，不保證列序。

下一句是獨立查詢；a 不能沿用。

```
SELECT * FROM a;
```

實際：no such table: a
表別名只在宣告它的那一句有效。

本頁重點：仍是八位。沒有新增名為 a 的表，也沒有更改原表 StudentId 欄名。

漏寫配對條件：八位學生配八筆選課

起點：第3、4頁的完整學校資料，八位學生、八筆選課。目標仍是每筆選課查正確姓名。A 不比較兩表學號；先預測八位乘八筆會有幾列。

```
SELECT s.StudentId,s.Name,e.CourseCode
FROM STUDENT AS s,ENROLLMENT AS e;
```

列 / 欄	E1	E2	E3	E4	E5	E6	E7	E8
S1	S1/E1	S1/E2	S1/E3	S1/E4	S1/E5	S1/E6	S1/E7	S1/E8
S2	S2/E1	S2/E2	S2/E3	S2/E4	S2/E5	S2/E6	S2/E7	S2/E8
S3	S3/E1	S3/E2	S3/E3	S3/E4	S3/E5	S3/E6	S3/E7	S3/E8
S5	S5/E1	S5/E2	S5/E3	S5/E4	S5/E5	S5/E6	S5/E7	S5/E8
S6	S6/E1	S6/E2	S6/E3	S6/E4	S6/E5	S6/E6	S6/E7	S6/E8
S7	S7/E1	S7/E2	S7/E3	S7/E4	S7/E5	S7/E6	S7/E7	S7/E8
S8	S8/E1	S8/E2	S8/E3	S8/E4	S8/E5	S8/E6	S8/E7	S8/E8
S9	S9/E1	S9/E2	S9/E3	S9/E4	S9/E5	S9/E6	S9/E7	S9/E8

完整64個配對。欄 E1–E8 指第4頁的原列；每格是一列輸出。例如 S3/E1 輸出 S3、Casey、DB101，並不表示 Casey 修 DB101。

本頁重點：8×8 = 64列；漏掉學號配對後，每位學生都被接上每一筆選課。

補上學號條件：只留八筆正確選課

起點：第3、4頁的完整學校資料，八位學生、八筆選課。B 只比 A 多一行學號相等；原資料與輸出欄相同。

先預測結果，再執行。

```
SELECT s.StudentId,s.Name,e.CourseCode
FROM STUDENT AS s,ENROLLMENT AS e
WHERE s.StudentId=e.StudentId;
```

StudentId	Name	CourseCode
S1	Alex	DB101
S1	Alex	CS102
S2	Alex	DB101
S6	Riley	CS102
S6	Riley	NW103
S7	Taylor	DB101
S8	Jordan	DB101
S9	Robin	CS102

圖按學號整理方便對照；SQL 未寫 ORDER BY，不保證列序。

本頁重點：只留下八筆。S3、S5 沒有選課，不在結果中；同名的 S1、S2 各用學號配對。

同表查詢：a、b 都來自同一張學生表

教務處要安排同系雙人討論，每對只列一次。先看 a、b 兩個位置可放誰，再逐步加條件。以下只做 SELECT，可以接續執行，不必每頁重設。

a：第一位學生

學號	姓名	系所
S1	Alex	D1
S2	Alex	D1
S3	Casey	NULL
S5	Morgan	D1
S6	Riley	D2
S7	Taylor	D2
S8	Jordan	D2
S9	Robin	NULL

b：第二位學生

學號	姓名	系所
S1	Alex	D1
S2	Alex	D1
S3	Casey	NULL
S5	Morgan	D1
S6	Riley	D2
S7	Taylor	D2
S8	Jordan	D2
S9	Robin	NULL

完整同一份八人資料展示兩次，方便比較；資料庫仍只有一張 STUDENT。

FROM STUDENT AS a, STUDENT AS b：兩次引用學生表，分別暫稱 a、b。
a.StudentId 與 b.StudentId 指兩個位置的學號；可以碰巧是同一人。

問題：S1 和 S2 都叫 Alex，能當成同一個人嗎？

回答：不能。用不同學號區分；本題每對由兩個學號決定。

本頁重點：同一張表與自己配對，稱同表接合（self-join）；AS 不會建立或複製資料表。

只寫 FROM：八位配八位，共64組

起點：第3、4頁的完整學校資料，八位學生、八筆選課。先不寫 WHERE。每位 a 都配八位 b；預測 S1/S1、S1/S2、S2/S1 是否都在結果中。

```
SELECT a.StudentId AS AId,b.StudentId AS BId
FROM STUDENT AS a,STUDENT AS b;
```

列 / 欄	S1	S2	S3	S5	S6	S7	S8	S9
S1	S1/S1	S1/S2	S1/S3	S1/S5	S1/S6	S1/S7	S1/S8	S1/S9
S2	S2/S1	S2/S2	S2/S3	S2/S5	S2/S6	S2/S7	S2/S8	S2/S9
S3	S3/S1	S3/S2	S3/S3	S3/S5	S3/S6	S3/S7	S3/S8	S3/S9
S5	S5/S1	S5/S2	S5/S3	S5/S5	S5/S6	S5/S7	S5/S8	S5/S9
S6	S6/S1	S6/S2	S6/S3	S6/S5	S6/S6	S6/S7	S6/S8	S6/S9
S7	S7/S1	S7/S2	S7/S3	S7/S5	S7/S6	S7/S7	S7/S8	S7/S9
S8	S8/S1	S8/S2	S8/S3	S8/S5	S8/S6	S8/S7	S8/S8	S8/S9
S9	S9/S1	S9/S2	S9/S3	S9/S5	S9/S6	S9/S7	S9/S8	S9/S9

完整64列排成矩陣：橫列是 a，直欄是 b。S1/S2 表示 AId=S1、BId=S2；下一頁只留下 a=S1 的一整橫列。

所有組合稱笛卡兒積（Cartesian product）。矩陣是完整結果的展示方式，不表示資料庫必須逐格執行。

本頁重點：答案：三組都在。每位 a 配8位 b，合計8×8 = 64；還沒有同系或不同人的限制。

加入 WHERE：固定 S1，留下8組

起點：第3、4頁的完整學校資料，八位學生、八筆選課。只加 WHERE a.StudentId='S1'。a 固定，b 仍可取全部八人；預測 S1/S1 是否會被排除。

先預測結果，再執行。

```
SELECT a.StudentId AS AId,b.StudentId AS BId
FROM STUDENT AS a,STUDENT AS b
WHERE a.StudentId='S1';
```

AId	BId
S1	S1
S1	S2
S1	S3
S1	S5
S1	S6
S1	S7
S1	S8
S1	S9

圖按學號整理方便對照；SQL 未寫 ORDER BY，不保證列序。

本頁重點：64組留下8組，即1×8。S1/S1 仍在；只固定第一位，不會自動排除本人或不同系的人。

先顯示同系判斷：8組仍全部列出

起點：第3、4頁的完整學校資料，八位學生、八筆選課。a 固定 S1（D1）。只在 SELECT 加上 b.DeptId 與同系判斷 SameDept，還沒用同系條件篩選。先猜 S3、S9 的判斷值。

先預測結果，再執行。

```
SELECT a.StudentId AS AId,b.StudentId AS BId,
       b.DeptId,a.DeptId=b.DeptId AS SameDept
FROM STUDENT AS a,STUDENT AS b
WHERE a.StudentId='S1';
```

AId	BId	DeptId	SameDept
S1	S1	D1	1
S1	S2	D1	1
S1	S3	NULL	NULL
S1	S5	D1	1
S1	S6	D2	0
S1	S7	D2	0
S1	S8	D2	0
S1	S9	NULL	NULL

圖按學號整理方便對照；SQL 未寫 ORDER BY，不保證列序。

本頁重點：三列為1、三列為0、兩列為NULL。SQLite 分別表示 TRUE / FALSE / UNKNOWN；SELECT 顯示判斷，不會因此篩掉任何列。

加入同系條件：S1 的8組剩3組

回到只輸出 AId、BId 的查詢；在原 WHERE 加 AND a.DeptId=b.DeptId。AND 表示兩個條件都要成立。先從前頁的判斷表找出會留下的 b。

```
SELECT a.StudentId AS AId,b.StudentId AS BId
FROM STUDENT AS a,STUDENT AS b
WHERE a.StudentId='S1'
  AND a.DeptId=b.DeptId;
```

AId	BId
S1	S1
S1	S2
S1	S5

實際完整結果三列；S1 本人仍在。

b 學號	前頁 SameDept	本次 WHERE
S1、S2、S5	1	保留3列
S6、S7、S8	0	排除3列
S3、S9	NULL	排除2列

逐列判斷的整理：WHERE 只留1；0與NULL都不保留。

問題：為何不是只剩 S2、S5？

回答：S1 的 D1 和自己相等；這一步尚未要求學號不同。

本頁重點：8 - 3 - 2 = 3組。同系不代表不同人；S1/S1 是下一步要排除的反例。

加入學號不同：S1 的3組剩2組

目標是找能與 S1 討論的其他同系同學。在前頁程式只加 AND a.StudentId<>b.StudentId；<> 表示不相等。先預測哪一列會被排除。

```
SELECT a.StudentId AS AId,b.StudentId AS BId
FROM STUDENT AS a,STUDENT AS b
WHERE a.StudentId='S1'
  AND a.DeptId=b.DeptId
  AND a.StudentId<>b.StudentId;
```

AId	BId
S1	S2
S1	S5

實際完整結果兩列；S1/S1 被排除。

前頁候選	學號是否不同	這一步的結果
S1 / S1	否	排除
S1 / S2	是	保留
S1 / S5	是	保留

同一問題的錯誤 / 修正：只要求同系會包含本人，加學號不同才符合「其他同學」。

問題：兩位都叫 Alex，S1/S2 為何仍保留？

回答：因為學號不同。排除本人要比學號，不能改成 a.Name<>b.Name。

本頁重點：完整過程：64組→固定 S1 留8組→同系留3組→排除本人留2組。原學生表一直是8列。

回到全班：只用同系條件，留下18組

先回到第22頁沒有 WHERE 的64組，不再固定 S1，也先不排除本人。這次只加同系條件，讓每位學生都有被配對的機會。

```
SELECT a.StudentId AS AId,b.StudentId AS BId
FROM STUDENT AS a,STUDENT AS b
WHERE a.DeptId=b.DeptId;
```

列 / 欄	S1	S2	S3	S5	S6	S7	S8	S9
S1	S1/S1	S1/S2	-	S1/S5	-	-	-	-
S2	S2/S1	S2/S2	-	S2/S5	-	-	-	-
S3	-	-	-	-	-	-	-	-
S5	S5/S1	S5/S2	-	S5/S5	-	-	-	-
S6	-	-	-	-	S6/S6	S6/S7	S6/S8	-
S7	-	-	-	-	S7/S6	S7/S7	S7/S8	-
S8	-	-	-	-	S8/S6	S8/S7	S8/S8	-
S9	-	-	-	-	-	-	-	-

完整18組：D1 的3×3 = 9，D2 的3×3 = 9。- 表示未留下；S3、S9 尚未分系，不因此算同系。

問題：六個對角線配對要保留嗎？

回答：若目標是雙人討論，就不應保留。S1/S1 等六組仍是本人配本人；下一頁加學號不同。

本頁重點：9 + 9 = 18組。這不是接續前頁的兩組再新增資料，而是重新查詢同一份八人原表。

不同學生：18組扣六組自我配對

起點：第3、4頁的完整學校資料，八位學生、八筆選課。相同系所之外，只加學號不同；先預測18扣6的結果。

先預測結果，再執行。

```
SELECT a.StudentId AS AId,b.StudentId AS BId
FROM STUDENT AS a,STUDENT AS b
WHERE a.DeptId=b.DeptId AND a.StudentId<>b.StudentId;
```

AId	BId
S1	S2
S1	S5
S2	S1
S2	S5
S5	S1
S5	S2
S6	S7
S6	S8
S7	S6
S7	S8
S8	S6
S8	S7

圖按學號整理方便對照；SQL 未寫 ORDER BY，不保證列序。

本頁重點：留下12列。只扣六位已分系者；S3、S9 原就不在同系結果。S1/S2 與 S2/S1 都保留。

雙人討論：每對學生只列一次

起點：第3、4頁的完整學校資料，八位學生、八筆選課。雙人討論不區分左 / 右。A 是前面的12列；B 只把 <> 換成 <，預測是否剩6對。

```
SELECT a.StudentId AS AId,b.StudentId AS BId
FROM STUDENT AS a,STUDENT AS b
WHERE a.DeptId=b.DeptId AND a.StudentId<b.StudentId;
```

A：<>，12列

AId	BId
S1	S2
S1	S5
S2	S1
S2	S5
S5	S1
S5	S2
S6	S7
S6	S8
S7	S6
S7	S8
S8	S6
S8	S7

B：<，6列

AId	BId
S1	S2
S1	S5
S2	S5
S6	S7
S6	S8
S7	S8

< 只留學號較小的一位在左；不表示年齡或優先權。每對反向只留一列。

本頁重點：同系雙人討論需6對。只有每對只需一次時，才用 < 去除反向組合。

讀者與作者：兩個方向都要保留

起點：第3、4頁的完整學校資料，八位學生、八筆選課。新任務：每位學生閱讀所有其他同系同學的報告。a 是讀者，b 是作者；S1 讀 S2 與 S2 讀 S1 是兩項任務。

A：用 <，預測哪些讀者的工作會漏掉。

```
SELECT a.StudentId AS ReaderId,b.StudentId AS AuthorId
FROM STUDENT AS a,STUDENT AS b
WHERE a.DeptId=b.DeptId AND a.StudentId<b.StudentId;
```

B：只把 < 改為 <>。

```
SELECT a.StudentId AS ReaderId,b.StudentId AS AuthorId
FROM STUDENT AS a,STUDENT AS b
WHERE a.DeptId=b.DeptId AND a.StudentId<>b.StudentId;
```

A：<，漏6項

ReaderId	AuthorId
S1	S2
S1	S5
S2	S5
S6	S7
S6	S8
S7	S8

B：<>，完整12項

ReaderId	AuthorId
S1	S2
S1	S5
S2	S1
S2	S5
S5	S1
S5	S2
S6	S7
S6	S8
S7	S6
S7	S8
S8	S6
S8	S7

A 漏掉反向閱讀，如 S2→S1；B 符合同系互讀。兩段只差 < 與 <>，未分系者本題不配對。

本頁重點：本題答案12項，不是6項；方向不同代表不同工作，不能當重複刪掉。

DISTINCT：去除相同輸出，不是合併同名學生

起點：第3、4頁的完整學校資料，八位學生、八筆選課。先看每位學生姓名，再試去重。兩位 Alex 不同人，但姓名字串相同。

```
SELECT Name FROM STUDENT;
```

```
SELECT DISTINCT Name FROM STUDENT;
```

A：8列姓名

Name
Alex
Alex
Casey
Morgan
Riley
Taylor
Jordan
Robin

B：7個不同姓名

Name
Alex
Casey
Morgan
Riley
Taylor
Jordan
Robin

DISTINCT 看輸出欄是否完全相同；兩個 Alex 在 B 合成一個姓名值，不代表學生剩七位。

問題：要列不重複的學生，應保留什麼？

回答：保留 StudentId，不能只靠 Name。

本頁重點：去除相同姓名值與指出每位學生不是同一件事。

DISTINCT：反向配對仍是不同的列

起點：第3、4頁的完整學校資料，八位學生、八筆選課。目標回到同系雙人討論，每對一次。兩段都有 DISTINCT，只差配對條件；預測12列加 DISTINCT 是否變6列。

```
SELECT DISTINCT a.StudentId AS AId,b.StudentId AS BId
FROM STUDENT AS a,STUDENT AS b
WHERE a.DeptId=b.DeptId AND a.StudentId<>b.StudentId;
```

```
SELECT DISTINCT a.StudentId AS AId,b.StudentId AS BId
FROM STUDENT AS a,STUDENT AS b
WHERE a.DeptId=b.DeptId AND a.StudentId<b.StudentId;
```

A：DISTINCT，12列

AId	BId
S1	S2
S1	S5
S2	S1
S2	S5
S5	S1
S5	S2
S6	S7
S6	S8
S7	S6
S7	S8
S8	S6
S8	S7

B：<，6列

AId	BId
S1	S2
S1	S5
S2	S5
S6	S7
S6	S8
S7	S8

S1/S2 與 S2/S1 的整列值不同；DISTINCT 不把反向當成相同。

本頁重點：每對一次要改配對條件；加 DISTINCT 不會把12個不同輸出合成6列。

同系同學名單：先保留方向，再去重

起點：第3、4頁的完整學校資料，八位學生、八筆選課。現在要列有同系同學的每位學生，不是列每對。預期 D1 三人、D2 三人，共6位。

```
SELECT DISTINCT a.StudentId,a.Name
FROM STUDENT AS a,STUDENT AS b
WHERE a.DeptId=b.DeptId AND a.StudentId<b.StudentId;
```

```
SELECT DISTINCT a.StudentId,a.Name
FROM STUDENT AS a,STUDENT AS b
WHERE a.DeptId=b.DeptId AND a.StudentId<>b.StudentId;
```

A：<，只有4人

StudentId	Name
S1	Alex
S2	Alex
S6	Riley
S7	Taylor

B：<>，完整6人

StudentId	Name
S1	Alex
S2	Alex
S5	Morgan
S6	Riley
S7	Taylor
S8	Jordan

A 沒有 S5、S8，他們只在右邊；B 保留方向再去重。兩段未寫 ORDER BY，不保證列序。

本頁重點：列人與列配對要分清楚。B 列完整六位學生；同名 Alex 因學號不同仍是兩列。

UNION：合併兩門課的學生名單

起點：第3、4頁的完整學校資料，八位學生、八筆選課。兩份名單共有7筆，S1 兩邊都有。先預測直接合併與去重合併各有幾列。

課程	完整學生名單
DB101	S1、S2、S7、S8
CS102	S1、S6、S9

```
SELECT StudentId FROM ENROLLMENT WHERE CourseCode='DB101'
UNION ALL
SELECT StudentId FROM ENROLLMENT WHERE CourseCode='CS102';
```

```
SELECT StudentId FROM ENROLLMENT WHERE CourseCode='DB101'
UNION
SELECT StudentId FROM ENROLLMENT WHERE CourseCode='CS102';
```

A：UNION ALL，7列

StudentId
S1
S2
S7
S8
S1
S6
S9

B：UNION，6列

StudentId
S1
S2
S6
S7
S8
S9

A 保留 S1 兩次，B 去重；S5 未選課不出現。未寫 ORDER BY，不保證列序。

本頁重點：UNION ALL 保留兩份結果的重複；UNION 去除相同輸出列。

INTERSECT：兩門課都修的學生

起點：第3、4頁的完整學校資料，八位學生、八筆選課。兩份完整名單如下；先預測共同學號。

課程	完整學生名單
DB101	S1、S2、S7、S8
CS102	S1、S6、S9

先預測結果，再執行。

```
SELECT StudentId FROM ENROLLMENT WHERE CourseCode='DB101'
INTERSECT
SELECT StudentId FROM ENROLLMENT WHERE CourseCode='CS102';
```

StudentId
S1

圖按學號整理方便對照；SQL 未寫 ORDER BY，不保證列序。

本頁重點：只有 S1 同時出現在 DB101 與 CS102 名單；成績是否登錄不是本題條件。

EXCEPT：課程名單相減有方向

起點：第3、4頁的完整學校資料，八位學生、八筆選課。只修 DB101、不修 CS102，與只修 CS102、不修 DB101，是兩個不同問題。

課程	完整學生名單
DB101	S1、S2、S7、S8
CS102	S1、S6、S9

```
SELECT StudentId FROM ENROLLMENT WHERE CourseCode='DB101'
EXCEPT
SELECT StudentId FROM ENROLLMENT WHERE CourseCode='CS102';
```

```
SELECT StudentId FROM ENROLLMENT WHERE CourseCode='CS102'
EXCEPT
SELECT StudentId FROM ENROLLMENT WHERE CourseCode='DB101';
```

DB101 減 CS102

StudentId
S2
S7
S8

CS102 減 DB101

StudentId
S6
S9

左 S2、S7、S8；右 S6、S9。S6 另修 NW103 不影響本題。未寫 ORDER BY，不保證列序。

本頁重點：差集有方向；這裡只比較指定的兩門課，不代表學生總共只修一門課。

集合運算：兩邊欄數要相同

起點：第3、4頁的完整學校資料，八位學生、八筆選課。教務處仍要合併學生學號名單。A 一邊一欄、另一邊兩欄；先預測可否合併。

```
SELECT StudentId FROM ENROLLMENT WHERE CourseCode='DB101'
UNION
SELECT StudentId,CourseCode FROM ENROLLMENT WHERE CourseCode='CS102';
```

A：欄數不同，查詢失敗
一欄學號不能直接與兩欄學號 / 課號合併。

```
SELECT StudentId FROM ENROLLMENT WHERE CourseCode='DB101'
UNION
SELECT StudentId FROM ENROLLMENT WHERE CourseCode='CS102';
```

StudentId
S1
S2
S6
S7
S8
S9

未寫 ORDER BY，不保證列序。

本頁重點：兩邊欄數要相同；即使欄數相同，也要確認兩邊都在表示學生學號。

LIKE：A 開頭與 A 後面恰一個字元

起點：第3、4頁的完整學校資料，八位學生、八筆選課。目標是姓名以 A 開頭。A_ 的底線代表恰一個字元；A% 的百分號代表任意長度。先預測 Alex 符合哪一個。

```
SELECT StudentId,Name FROM STUDENT WHERE Name LIKE 'A_';
```

```
SELECT StudentId,Name FROM STUDENT WHERE Name LIKE 'A%';
```

A：A_，0列
Alex 在 A 之後有 lex 三個字元，不符合 A_。

StudentId	Name
S1	Alex
S2	Alex

B 符合 A 開頭的目標，兩位 Alex 都留下。

本頁重點：_ 比對恰一個字元；% 比對任意長度。不是看到 A 就用 A_。

SELECT：預覽成績加5，不更改原表

起點：第3、4頁的完整學校資料，八位學生、八筆選課。只預覽0到80分的選課各加5；先預測 Grade 與 Preview 會是哪四列。

先預測結果，再執行。

```
SELECT StudentId,CourseCode,Grade,Grade+5 AS Preview
FROM ENROLLMENT WHERE Grade BETWEEN 0 AND 80;
```

StudentId	CourseCode	Grade	Preview
S1	DB101	80	85
S2	DB101	0	5
S7	DB101	60	65
S8	DB101	75	80

圖按學號整理方便對照；SQL 未寫 ORDER BY，不保證列序。

本頁重點：80→85、0→5、60→65、75→80。BETWEEN 含0與80；NULL 不符合。SELECT 計算不把新分數寫回。

ORDER BY：先排姓名，再排學號

起點：第3、4頁的完整學校資料，八位學生、八筆選課。同名時還要指定學號次序。ASC 是升冪，DESC 是降冪；預測兩位 Alex 誰先。

先預測結果，再執行。

```
SELECT StudentId,Name FROM STUDENT
ORDER BY Name ASC,StudentId DESC;
```

StudentId	Name
S2	Alex
S1	Alex
S3	Casey
S8	Jordan
S5	Morgan
S6	Riley
S9	Robin
S7	Taylor

結果已依程式的 ORDER BY 排列。

本頁重點：姓名升冪，同名學號降冪；S2 Alex 在 S1 Alex 前。排序只改結果顯示，不更改原表。

INSERT A：先加學生會找不到系所

練習新增 S1 Alex，所屬系 D1。A 另開新資料庫，依第6、9、10頁建四張空表並開外來鍵，尚未載入任何資料。



箭頭是 STUDENT 參照 DEPT。INSERT INTO 指定表與欄位；VALUES 依欄位順序填值。

```
INSERT INTO STUDENT(StudentId,Name,DeptId)
VALUES ('S1','Alex','D1');
```

實際：FOREIGN KEY constraint failed
D1 尚不存在；S1 沒加入，四張表仍空。

本頁重點：這筆學生的外來鍵參照 D1；要先加入系所，不是重試相同學生新增。

INSERT B：先加系所，再加學生

B 另用相同四張空表；仍新增同一筆 S1。只改新增順序，不沿用其他練習資料。依次執行第1、2句。

```
INSERT INTO DEPT(DeptId,DeptName) VALUES ('D1','資訊管理');
INSERT INTO STUDENT(StudentId,Name,DeptId)
VALUES ('S1','Alex','D1');
```

執行後	DEPT	STUDENT
第1句	D1	0列
第2句	D1	S1 / Alex / D1

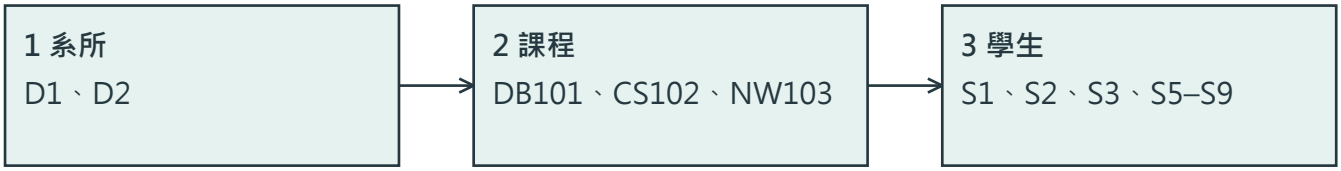
問題：這與 A 的輸入有什麼不同？

回答：學生的三個值完全相同；只有 B 先加入它參照的 D1。COURSE、ENROLLMENT 仍空。

本頁重點：先父表，再子表；新學生不會替你自動建立 D1。

INSERT：載入系所、課程與八位學生

完整資料重設第1步：另開新的無表資料庫，依第6、9、10頁建四張空表並開外來鍵。不要接續 S1 新增練習。



箭頭是執行順序，不是參照方向。由上而下執行；省略欄位清單時，依建表欄位順序填值。

```
INSERT INTO DEPT VALUES ('D1','資訊管理'),('D2','會計');
INSERT INTO COURSE VALUES ('DB101'),('CS102'),('NW103');
INSERT INTO STUDENT(StudentId,Name,DeptId) VALUES
  ('S1','Alex','D1'),('S2','Alex','D1'),
  ('S3','Casey',NULL),('S5','Morgan','D1'),
  ('S6','Riley','D2'),('S7','Taylor','D2'),
  ('S8','Jordan','D2'),('S9','Robin',NULL);
```

DEPT	COURSE	STUDENT	ENROLLMENT
2	3	8	0

本頁重點：此時兩系、三課、八位學生都已加入；選課仍空，下一步才加入。

INSERT：載入八筆選課

完整資料重設第2步：接續第43頁同一資料庫。八位學生與三門課都存在；只新增選課，不重做 CREATE TABLE 或前面的 INSERT。

```
INSERT INTO ENROLLMENT(StudentId,CourseCode,Grade) VALUES
  ('S1','DB101',80),('S1','CS102',NULL),
  ('S2','DB101',0),('S6','CS102',90),
  ('S7','DB101',60),('S8','DB101',75),
  ('S9','CS102',NULL),('S6','NW103',95);
```

StudentId	CourseCode	Grade
S1	DB101	80
S1	CS102	NULL
S2	DB101	0
S6	CS102	90
S7	DB101	60
S8	DB101	75
S9	CS102	NULL
S6	NW103	95

實際全部成功；完整資料與第4頁相同，沒有 E1–E8 欄位。

問題：為什麼不能先加選課，再加學生與課程？

回答：每筆選課的學號與課號都要能查到已存在的父表列。

本頁重點：重設完成：DEPT 2列、COURSE 3列、STUDENT 8列、ENROLLMENT 8列。

DEFAULT：省略 DeptId 與明填 NULL

新學生 S4 Dana 尚未分系，目標是 DeptId 為 NULL。A、B 各另開空資料庫：依第6、9、10頁建表，再依第43、44頁載入；兩邊起點都尚無 S4。

DeptId DEFAULT 'D1' 表示省略時填 D1，不是 NULL。A 只填學號與姓名。

```
INSERT INTO STUDENT(StudentId,Name) VALUES ('S4','Dana');
SELECT * FROM STUDENT WHERE StudentId='S4';
```

B 只改成明填 NULL，預測是否仍套用 D1。

```
INSERT INTO STUDENT(StudentId,Name,DeptId) VALUES ('S4','Dana',NULL);
SELECT * FROM STUDENT WHERE StudentId='S4';
```

A：與目標不同

StudentId	Name	DeptId
S4	Dana	D1

B：符合尚未分系

StudentId	Name	DeptId
S4	Dana	NULL

本頁重點：省略欄位會套 DEFAULT；明填 NULL 不套 DEFAULT。本題 B 才記錄尚未分系。

CREATE TABLE：建立修課學號名單空表

起點：第3、4頁的完整學校資料，八位學生、八筆選課。教務處要列每位修課學生一次。選課有八列，但 S1、S6 各有兩列；學生名單不是八位。

StudentId	選課筆數
S1	2
S2	1
S3	0
S5	0
S6	2
S7	1
S8	1
S9	1

這是由完整原資料逐筆整理的對照，不是本頁要學的新 SQL 語法。

第1步：目的表尚不存在。預測建表後是否就有修課學生。

```
CREATE TABLE STUDENT_LIST (StudentId TEXT NOT NULL PRIMARY KEY);
```

實際：STUDENT_LIST 仍0列
StudentId 是主鍵；建表不會執行選課查詢。

本頁重點：先建立空名單；下一步才用 INSERT SELECT 放入查詢結果。

INSERT SELECT：修課學號先去重再新增

接續第46頁的同一資料庫，STUDENT_LIST 已存在且0列。目標是 S1、S2、S6、S7、S8、S9 各一次。

A 直接新增八個學號，先預測目的表主鍵能否接受。

```
INSERT INTO STUDENT_LIST SELECT StudentId FROM ENROLLMENT;
```

A：主鍵重複，整句新增失敗
來源有兩次 S1、兩次 S6；目的表仍0列。

B 接續失敗後的同一空名單，只在 SELECT 加 DISTINCT；不重做 CREATE TABLE。

```
INSERT INTO STUDENT_LIST
SELECT DISTINCT StudentId FROM ENROLLMENT;
SELECT StudentId FROM STUDENT_LIST ORDER BY StudentId;
```

StudentId
S1
S2
S6
S7
S8
S9

本頁重點：B 加入六位修課學生；來源八筆選課不變，S3、S5 不會出現在修課名單。

CREATE TABLE AS：複製目前的成績

起點：第3、4頁的完整學校資料，八位學生、八筆選課。教務處要留目前成績副本，尚無 ENROLL_COPY。實作額外補充（SQLite）：AS 後接查詢，建立表時一起放入查詢結果。

```
CREATE TABLE ENROLL_COPY AS SELECT * FROM ENROLLMENT;
SELECT * FROM ENROLL_COPY ORDER BY StudentId,CourseCode;
```

StudentId	CourseCode	Grade
S1	CS102	NULL
S1	DB101	80
S2	DB101	0
S6	CS102	90
S6	NW103	95
S7	DB101	60
S8	DB101	75
S9	CS102	NULL

問題：與 CREATE TABLE 空表有何不同？

回答：這裡副本立刻有八列；原表仍八列，S2 的0分與兩個 NULL 都保留。

本頁重點：AS SELECT 複製這一次查詢的資料；是否複製主鍵限制，圖書館案例再比較。

DELETE：學號不足以指定一筆選課

起點：第3、4頁的完整學校資料，八位學生、八筆選課。只核准刪除 S1 / CS102 的誤登，S1 / DB101 的80分與其他選課要保留。A、B 各自另用相同原資料。

```
DELETE FROM ENROLLMENT WHERE StudentId='S1';
SELECT * FROM ENROLLMENT ORDER BY StudentId,CourseCode;
```

```
DELETE FROM ENROLLMENT
WHERE StudentId='S1' AND CourseCode='CS102';
SELECT * FROM ENROLLMENT ORDER BY StudentId,CourseCode;
```

A：誤刪兩列，剩6列

StudentId	CourseCode	Grade
S2	DB101	0
S6	CS102	90
S6	NW103	95
S7	DB101	60
S8	DB101	75
S9	CS102	NULL

B：只刪一列，剩7列

StudentId	CourseCode	Grade
S1	DB101	80
S2	DB101	0
S6	CS102	90
S6	NW103	95
S7	DB101	60
S8	DB101	75
S9	CS102	NULL

本頁重點：DELETE FROM 選表；WHERE 選列。本題要學號與課號都寫，才不會誤刪另一門課。

UPDATE：只更正指定選課的成績

起點：第3、4頁的完整學校資料，八位學生、八筆選課。只核准 S1 / DB101 的80改85；CS102 的 NULL 要保留。A、B 各自從原資料開始。

```
UPDATE ENROLLMENT SET Grade=85 WHERE StudentId='S1';
SELECT * FROM ENROLLMENT ORDER BY StudentId,CourseCode;
```

```
UPDATE ENROLLMENT SET Grade=85
WHERE StudentId='S1' AND CourseCode='DB101';
SELECT * FROM ENROLLMENT ORDER BY StudentId,CourseCode;
```

A：兩筆都85

StudentId	CourseCode	Grade
S1	CS102	85
S1	DB101	85
S2	DB101	0
S6	CS102	90
S6	NW103	95
S7	DB101	60
S8	DB101	75
S9	CS102	NULL

B：只改 DB101

StudentId	CourseCode	Grade
S1	CS102	NULL
S1	DB101	85
S2	DB101	0
S6	CS102	90
S6	NW103	95
S7	DB101	60
S8	DB101	75
S9	CS102	NULL

本頁重點：SET 說改成什麼；WHERE 說改哪一列。本題 B 才只更正核准的一筆。

成績副本：原表更正不會同步

起點：第3、4頁的完整學校資料，八位學生、八筆選課。另重設完整原資料，尚無 ENROLL_COPY；不接續第48頁的副本或85分測試。核准將 S2 / DB101 的0改5；以下三步接續同一資料庫。

1 複製原成績，2 只改原表，3 分別查原表、副本。先預測會看到哪兩個值。

```
CREATE TABLE ENROLL_COPY AS SELECT * FROM ENROLLMENT;
UPDATE ENROLLMENT SET Grade=5
WHERE StudentId='S2' AND CourseCode='DB101';
SELECT Grade FROM ENROLLMENT WHERE StudentId='S2';
SELECT Grade FROM ENROLL_COPY WHERE StudentId='S2';
```

先查原表

Grade
5

再查副本

Grade
0

原表只有 S2 的成績改了；另外七筆不變，副本八筆全保留複製時的值。

本頁重點：副本保留複製當下的0分；它不是會自動隨原表更新的查詢。

CASCADE 比較：先看完整參照鏈

起點：第3、4頁的完整學校資料，八位學生、八筆選課。接著用同一份人工資料，比較改系所代碼與核准刪系所。不要接續85分、副本或 S4 新增練習。



箭頭是參照方向：選課找學生、學生找系所。下一頁再讀每段外來鍵的動作設定。

StudentId	DeptId	選課筆數
S1	D1	2
S2	D1	1
S3	NULL	0
S5	D1	0
S6	D2	2
S7	D2	1
S8	D2	1
S9	NULL	1

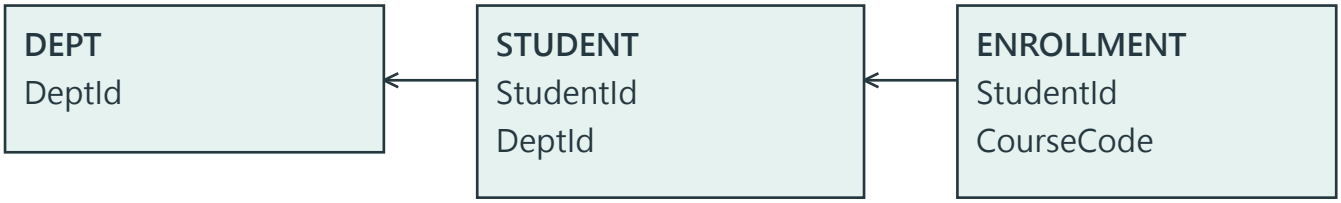
問題：D1 的三位學生，選課分別幾筆？

回答：S1 兩筆、S2 一筆、S5 零筆。S9 未分系但有一筆選課；這些關係不能混為一談。

本頁重點：系所影響學生的 DeptId；選課參照的是 StudentId，不是系所代碼。

外來鍵動作：設定位置與操作方向

起點：第3、4頁的完整學校資料，八位學生、八筆選課。這一組只先讀設定，還沒更改或刪除。預設版保留學生與選課，刪除測試會另外明確改需求與設定。



STUDENT.DeptId 的設定

UPDATE: CASCADE / DELETE: RESTRICT

ENROLLMENT.StudentId 的設定

DELETE: NO ACTION

箭頭是外來鍵的參照方向：選課參照學生，學生參照系所。連動操作由被參照的父表影響子表。

子表欄位	改父表代碼	刪父表列
STUDENT.DeptId	CASCADE：跟著改 DeptId	RESTRICT：有學生時拒絕
ENROLLMENT.StudentId	預設 NO ACTION	預設 NO ACTION

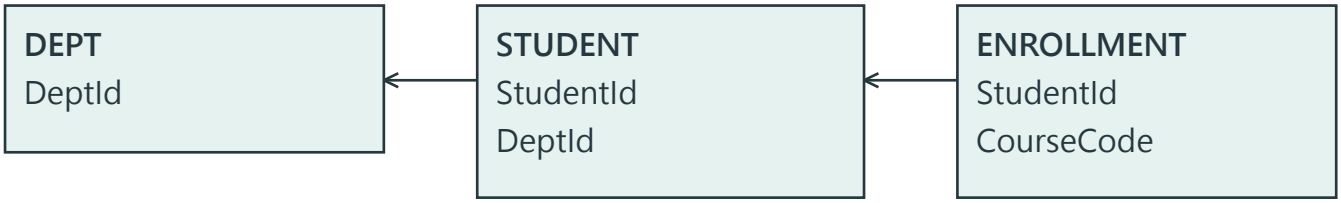
若仍有選課參照要刪的學生，本例 NO ACTION 會拒絕整句刪除。

實作額外補充（SQLite）：本章使用預設檢查時點；NO ACTION 在語句結束時檢查，RESTRICT 在修改父表列時檢查，不能視為完全相同。

本頁重點：設定放在參照父表的子表。更新同步欄值；刪除能否連動，要看整條鏈的刪除設定。

UPDATE A：先改學生會找不到新系所

起點：第3、4頁的完整學校資料，八位學生、八筆選課。教務處要將 D1 代碼改 D10，所有學生與成績仍保留。A 先改學生，D10 尚不存在。



STUDENT.DeptId 的設定

UPDATE: CASCADE / DELETE: RESTRICT

ENROLLMENT.StudentId 的設定

DELETE: NO ACTION

箭頭是外來鍵的參照方向：選課參照學生，學生參照系所。連動操作由被參照的父表影響子表。

```
UPDATE STUDENT SET DeptId='D10' WHERE DeptId='D1';
```

實際：FOREIGN KEY constraint failed
D1、D2 仍存在；八位學生、八筆選課全保留。
S1、S2、S5 的 DeptId 仍 D1，不會改到一半。

本頁重點：先改子表會參照不存在的 D10；下一頁改父表，讓 CASCADE 同步正確代碼。

ON UPDATE CASCADE：改父表並同步三位學生

接續 A 失敗後的資料，仍是完整原資料。只改父表 DEPT 的 D1；ON UPDATE CASCADE 已在 STUDENT 的正式定義中。

```
UPDATE DEPT SET DeptId='D10' WHERE DeptId='D1';
SELECT StudentId,DeptId FROM STUDENT ORDER BY StudentId;
```

StudentId	改前 DeptId	改後 DeptId
S1	D1	D10
S2	D1	D10
S3	NULL	NULL
S5	D1	D10
S6	D2	D2
S7	D2	D2
S8	D2	D2
S9	NULL	NULL

改前欄取自第3頁；SELECT 只輸出學號與改後值。

問題：為什麼八筆選課不用改 StudentId？

回答：這次只改系所代碼；學生的學號沒變，選課參照的值就沒變。

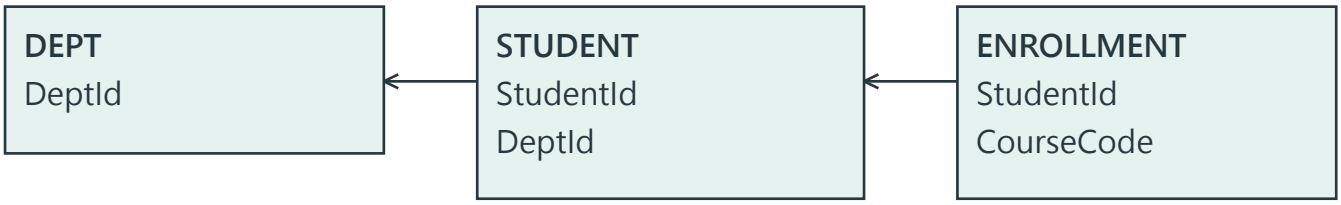
本頁重點：S1、S2、S5 的 DeptId 同步為 D10；其他五人與八筆選課都不變。

刪除 A 的準備：只有學生設 CASCADE

獨立刪除測試需求：假設已核准刪 D1 及其學生與選課，觀察連動設定是否完整。這不是正式學校資料的保留規則；不接續改成 D10 的資料庫。

新無表資料庫：完整執行第6頁 → 下方 STUDENT → 第10頁選課表 → 第43、44頁只執行 INSERT。不要依載入頁開頭再次建表；每段只執行一次。

```
CREATE TABLE STUDENT (
  StudentId TEXT NOT NULL PRIMARY KEY,
  Name TEXT NOT NULL,
  DeptId TEXT DEFAULT 'D1',
  FOREIGN KEY (DeptId) REFERENCES DEPT(DeptId)
    ON DELETE CASCADE ON UPDATE CASCADE
);
```



STUDENT.DeptId 的設定

UPDATE: CASCADE / DELETE: CASCADE

ENROLLMENT.StudentId 的設定

DELETE: NO ACTION

箭頭是外來鍵的參照方向：選課參照學生，學生參照系所。連動操作由被參照的父表影響子表。

本頁重點：A 只有 DEPT→STUDENT 的刪除能連動；ENROLLMENT.StudentId 仍未設 CASCADE。

刪除 A 的結果：選課外來鍵阻止整句刪除

接續 A 準備完成的八人、八筆選課。只執行 DELETE D1；預測 S5 沒有選課是否會先被刪掉。

```
DELETE FROM DEPT WHERE DeptId='D1';
```

實際：FOREIGN KEY constraint failed
選課仍參照 S1、S2，阻止這句刪除。
D1、S1、S2、S5 都保留；沒有只先刪掉 S5。

DEPT	COURSE	STUDENT	ENROLLMENT
2	3	8	8

失敗後逐表 SELECT * 核對；每張表的原列都保留。

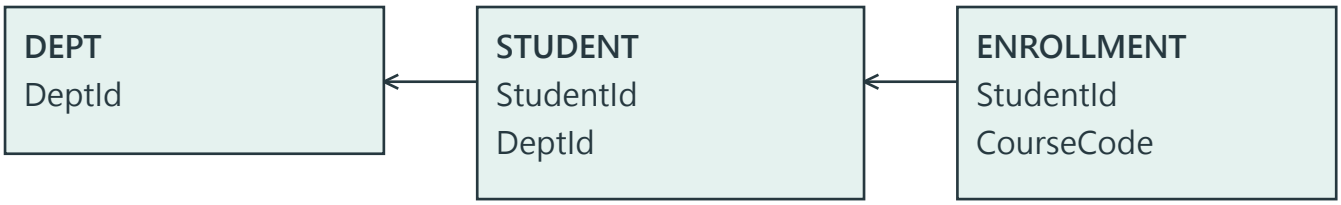
本頁重點：不是刪一部分再停；整句失敗，所有原列都保留。

刪除 B 的準備：選課也設 CASCADE

B 另開新的無表資料庫，使用與 A 相同學生、課程、八筆選課。唯一新增差異：ENROLLMENT.StudentId 也設 ON DELETE CASCADE。

順序：第6頁 → 第56頁 STUDENT 定義 → 下方 ENROLLMENT → 第43、44頁只執行 INSERT，不重做建表。外來鍵全程開啟。

```
CREATE TABLE ENROLLMENT (
  StudentId TEXT NOT NULL REFERENCES STUDENT(StudentId)
    ON DELETE CASCADE,
  CourseCode TEXT NOT NULL REFERENCES COURSE(CourseCode),
  Grade INTEGER,
  PRIMARY KEY (StudentId, CourseCode),
  CONSTRAINT grade_domain CHECK (Grade IS NULL OR
    (typeof(Grade) = 'integer' AND Grade BETWEEN 0 AND 100))
);
```



STUDENT.DeptId 的設定

UPDATE: CASCADE / DELETE: CASCADE

ENROLLMENT.StudentId 的設定

DELETE: CASCADE

箭頭是外來鍵的參照方向：選課參照學生，學生參照系所。連動操作由被參照的父表影響子表。

本頁重點：A、B 的資料與要刪的 D1 相同；B 只補上選課這一段刪除連動。

刪除 B 的結果：留下哪些學生與選課

接續 B 準備完成的八人、八筆選課。執行與 A 完全相同的 DELETE D1；先預測三位 D1 學生及其三筆選課會怎樣。

```
DELETE FROM DEPT WHERE DeptId='D1';
SELECT * FROM STUDENT;
SELECT * FROM ENROLLMENT;
```

StudentId	Name	DeptId
S3	Casey	NULL
S6	Riley	D2
S7	Taylor	D2
S8	Jordan	D2
S9	Robin	NULL

STUDENT：剩五位；DEPT 只剩 D2。

StudentId	CourseCode	Grade
S6	CS102	90
S7	DB101	60
S8	DB101	75
S9	CS102	NULL
S6	NW103	95

ENROLLMENT：剩五筆；兩句未寫 ORDER BY，不保證列序。

本頁重點：D1、S1、S2、S5 與三筆相關選課刪掉；保留五位學生、五筆選課，三門課全保留。

CASCADE：更新代碼與刪除資料比較

以下兩次操作各自從完整原資料開始，不先更新再刪除。更新版用正式設定；刪除版用剛才 B 的兩段 CASCADE。

同一原起點	更新 D1→D10	刪除 D1（B設定）
學生原8列	8列保留，3筆 DeptId 改值	剩5列，D1 的3人刪掉
選課原8列	8列全保留，學號未改	剩5列，相關3筆刪掉
課程原3列	3列全保留	3列全保留

問題：為什麼選課只刪三筆，不是每位 D1 學生各一筆？

回答：S1 有兩筆、S2 一筆、S5 沒選課，所以是2 + 1 + 0 = 3。依實際參照列，不依人數猜。

本頁重點：ON UPDATE CASCADE 同步所參照的代碼；ON DELETE CASCADE 刪除相關整列，不是把成績改 NULL。

圖書館案例：會員、實體書與每次借閱

圖書館記錄每位會員、每本實體書與每次借還。會員可未借過書，同名會員不合併；同書名可有不同實體。這個新故事再練建表、限制、查詢與修改。



箭頭：借閱列依代碼找到既有會員與實體書。

本例只有會員、實體書、借閱三種紀錄；書名相同也不代表同一本實體書。

LoanId 指一次借閱；同一會員可在不同日期再借同一本。圖書館要保留借閱歷史：有借閱紀錄時，不能直接刪會員。沒有 Email 用 NULL，不用空字串。

本頁重點：學校的核准刪除連動是獨立設定比較；圖書館的正式需求是保留歷史，不能照搬 CASCADE。

圖書館資料：四位會員與四本實體書

MemberId 是會員識別，Name 是姓名，Email 可缺；CopyId 是實體書識別，Title 是書名。人工 Email 僅供 SQL 測試，不是聯絡資料。

MemberId	Name	Email
M1	小安	NULL
M2	小文	wen@lib.test
M3	小安	ning@lib.test
M4	小青	NULL

CopyId	Title
B1	SQL入門
B2	SQL_練習
B3	DB設計
B4	SQL入門

問題：兩個小安與兩本 SQL入門，可以合成一筆嗎？

回答：不能。M1、M3 是不同會員；B1、B4 是不同實體書。B2 的書名底線是字面字元，後面會練 LIKE。

本頁重點：用識別碼分開同名人物與同書名實體；Email 缺值與空字串不同。

圖書館資料：六次借閱的完整起點

BorrowedOn 是借出日，ReturnedOn 是歸還日；NULL 表示尚未歸還。日期皆為2026年固定 YYYY-MM-DD 文字。這六筆是後面每題的共同起點。

LoanId	MemberId	CopyId	BorrowedOn	ReturnedOn
L1	M1	B1	2026-09-01	2026-09-05
L2	M1	B1	2026-09-10	2026-09-15
L3	M2	B2	2026-09-11	NULL
L4	M3	B3	2026-09-04	2026-09-08
L5	M2	B4	2026-09-06	2026-09-09
L6	M3	B1	2026-09-16	NULL

M1 在9/1借 B1、9/5還；9/10又借、9/15還。M3 在9/16才借 B1，時間沒有重疊。M4 完全沒有借閱；L3、L6 有借閱但尚未還。

問題：M1 + B1 可以識別這兩次借閱嗎？

回答：不能。L1、L2 的會員與書號相同，仍是兩次事件；要用不同 LoanId。

同一本實體不能同時借給兩人。本例資料已另外檢查借期；第66頁建表的日期 CHECK 只檢查單筆日期，不會自動防止跨筆借期重疊。

本頁重點：六筆借閱分別有身分、對象與日期；不要把沒借過與借了未還混為一談。

CREATE TABLE：會員表與可缺的 Email

圖書館實作第1步：新的無表 SQLite 資料庫，外來鍵需開啟。會員姓名必填，Email 可以沒有；有填時不能重複，也不能是空字串。

```
PRAGMA foreign_keys = ON;
CREATE TABLE MEMBER (
  MemberId TEXT NOT NULL PRIMARY KEY,
  Name TEXT NOT NULL,
  Email TEXT UNIQUE CHECK (Email IS NULL OR Email <> '')
);
```

輸入情況	本例限制
MemberId 重複	PRIMARY KEY 拒絕
Email 沒有	NULL 可存
Email 是空字串	CHECK 拒絕
非 NULL Email 重複	UNIQUE 拒絕

本頁重點：開啟檢查與指定外來鍵是不同步驟；這張會員表沒有參照其他表。

CREATE TABLE：實體書表

圖書館實作第2步：接續第64頁，會員表存在但仍空。建立 COPY；每本實體的 CopyId 唯一，Title 必填但可以重複。

```
CREATE TABLE COPY (
  CopyId TEXT NOT NULL PRIMARY KEY,
  Title TEXT NOT NULL
);
```

CopyId	Title
B1	SQL入門
B4	SQL入門

這是原資料中兩本同書名實體，還沒執行 INSERT。

問題：為什麼 Title 不寫 UNIQUE？

回答：圖書館有兩本 SQL入門；限制書名唯一會錯誤拒絕第二本實體。

本頁重點：書名不是實體書識別；建表後 COPY 仍0列。

CREATE TABLE：借閱表與日期限制

圖書館實作第3步：已有兩張空父表 MEMBER、COPY。每筆借閱都要找到會員與實體書；有歸還日時，不能早於借出日。

```
CREATE TABLE LOAN (
  LoanId TEXT NOT NULL PRIMARY KEY,
  MemberId TEXT NOT NULL REFERENCES MEMBER(MemberId)
    ON DELETE RESTRICT,
  CopyId TEXT NOT NULL REFERENCES COPY(CopyId),
  BorrowedOn TEXT NOT NULL, ReturnedOn TEXT,
  CHECK (ReturnedOn IS NULL OR ReturnedOn >= BorrowedOn)
);
```



兩個箭頭都由借閱指向父表；依會員與書號查找。LoanId 不用姓名或書名識別。

MemberId 的 ON DELETE RESTRICT 保留會員所連的借閱歷史。CopyId 未另寫刪除動作：本例有借閱參照時也不能直接刪實體書。兩欄都 NOT NULL。

問題：TEXT 日期為什麼可用 >= 比較？

回答：本例先限定固定 YYYY-MM-DD，文字順序與日期先後一致；CHECK 不會因此驗證所有輸入都是有效曆日。

本頁重點：歸還日可 NULL；已填日期才比較先後。固定格式與有效日期需先核對，不能靠這一項 CHECK 全包。

INSERT：先載入會員與實體書

圖書館完整重設：新無表資料庫 → 第64、65、66頁建表 → 本頁載入父表。三表原本都空，不接續其他測試。

```
INSERT INTO MEMBER VALUES
  ('M1','小安',NULL),('M2','小文','wen@lib.test'),
  ('M3','小安','ning@lib.test'),('M4','小青',NULL);
INSERT INTO COPY VALUES
  ('B1','SQL入門'),('B2','SQL_練習'),
  ('B3','DB設計'),('B4','SQL入門');
```

MEMBER	COPY	LOAN
4	4	0

問題：為什麼兩個 Email=NULL 沒被 UNIQUE 拒絕？

回答：本例 SQLite 的 UNIQUE 允許多個 NULL；非 NULL 值才不能重複。不要假定所有資料庫都相同。

本頁重點：會員四列、實體四列先存在；六次借閱下一步才加入。

INSERT：載入六次借閱

接續第67頁同一資料庫，三表已建好，會員與實體都有資料。按 LOAN 的五個欄位順序，一次加入六列。

```
INSERT INTO LOAN VALUES
  ('L1','M1','B1','2026-09-01','2026-09-05'),
  ('L2','M1','B1','2026-09-10','2026-09-15'),
  ('L3','M2','B2','2026-09-11',NULL),
  ('L4','M3','B3','2026-09-04','2026-09-08'),
  ('L5','M2','B4','2026-09-06','2026-09-09'),
  ('L6','M3','B1','2026-09-16',NULL);
```

MEMBER	COPY	LOAN
4	4	6

這組是第63頁完整資料。每筆會員、書號都查得到；L3、L6 的歸還日 NULL 可通過 CHECK。

本頁重點：圖書館共同起點完成；第73頁起各題回六筆原資料。下一組限制測試另用空資料庫。

NOT NULL：同一筆缺 Email 的會員

只測 M1 小安沒有 Email 的情況，不載入其他會員或借閱。A、B 各用新無表資料庫；同一筆 INSERT，只有 Email 的 NOT NULL 不同。

```
PRAGMA foreign_keys = ON;
CREATE TABLE MEMBER (
  MemberId TEXT NOT NULL PRIMARY KEY,
  Name TEXT NOT NULL,
  Email TEXT NOT NULL UNIQUE CHECK (Email IS NULL OR Email <> '')
);
INSERT INTO MEMBER(MemberId,Name) VALUES ('M1','小安');
```

A：NOT NULL constraint failed: MEMBER.Email
Email 被要求必填；M1 未加入。

```
PRAGMA foreign_keys = ON;
CREATE TABLE MEMBER (
  MemberId TEXT NOT NULL PRIMARY KEY,
  Name TEXT NOT NULL,
  Email TEXT UNIQUE CHECK (Email IS NULL OR Email <> '')
);
INSERT INTO MEMBER(MemberId,Name) VALUES ('M1','小安');
SELECT * FROM MEMBER;
```

MemberId	Name	Email
M1	小安	NULL

B 允許 NULL，M1 成功加入；這才符合 Email 可缺的需求。

本頁重點：Email 可缺就不加 NOT NULL。B 的 UNIQUE 與 CHECK 容許 NULL；已填值仍不得重複或為空字串。

日期 A：沒有 CHECK 會存入錯填的歸還日

只測 L3 的兩個日期，另用新的無表資料庫與 DATE_A，不改正式 LOAN。原事件9/11借出；錯填9/9歸還，目標是拒絕這筆。

```
CREATE TABLE DATE_A (
  LoanId TEXT NOT NULL PRIMARY KEY,
  BorrowedOn TEXT NOT NULL,ReturnedOn TEXT
);
INSERT INTO DATE_A VALUES ('L3','2026-09-11','2026-09-09');
SELECT * FROM DATE_A;
```

LoanId	BorrowedOn	ReturnedOn
L3	2026-09-11	2026-09-09

問題：BorrowedOn 必填、兩欄都宣告 TEXT，為何沒有錯誤？

回答：必填與型別宣告不檢查借還先後；兩個非空日期文字都可以存入。

本頁重點：A 存入成功卻違反故事規則；下一頁只多日期 CHECK，輸入不改。

日期 B：只加 CHECK 拒絕相同錯誤輸入

B 另用新無表資料庫，日期欄位與 A 相同，只加借還先後檢查。仍輸入 L3、9/11借出、9/9歸還。

```
CREATE TABLE DATE_B (
  LoanId TEXT NOT NULL PRIMARY KEY,
  BorrowedOn TEXT NOT NULL,ReturnedOn TEXT,
  CHECK (ReturnedOn IS NULL OR ReturnedOn>=BorrowedOn)
);
INSERT INTO DATE_B VALUES ('L3','2026-09-11','2026-09-09');
```

INSERT 回傳 CHECK constraint failed 後，另執行下方 SELECT；不重做建表。

```
SELECT * FROM DATE_B;
```

SELECT 完整結果：0 列
DATE_B 已建立；錯填資料被拒絕，表仍空。

問題：如果歸還日是 NULL 呢？

回答：允許尚未還書；條件明確容許 NULL，不把缺值當早於借出日。

實作額外補充（SQLite）：WHERE 只留 TRUE；CHECK 的結果若為 NULL 也會通過。但 typeof(NULL) 回傳文字 null，與 integer 比較為 FALSE，第8頁仍需 IS NULL 分支。

本頁重點：同一筆錯誤日期，A 接受、B 拒絕；差別只有日期 CHECK。

日期 B：加入核對後的正確歸還日

接續 B 的失敗結果：DATE_B 仍空，不重複建表。已核對 L3 實際9/16歸還；只更正輸入日期，再新增同一事件。

```
INSERT INTO DATE_B VALUES ('L3','2026-09-11','2026-09-16');
SELECT * FROM DATE_B;
```

LoanId	BorrowedOn	ReturnedOn
L3	2026-09-11	2026-09-16

問題：若只把9/9隨意改成9/20讓 CHECK 通過，就算正確嗎？

回答：不算。CHECK 只能擋先後顛倒；歸還日仍要依核對的實際事件填寫。

本頁重點：新增成功不等於任何日期都真實；本題要存已核對的9/16。正式 LOAN 仍用六筆原資料。

圖書館查詢 A：漏書號條件產生24列

起點：第62、63頁的完整圖書館資料，六筆借閱。目標是每次借閱查正確會員姓名、書名。A 已用會員代碼配對，但沒有比較借閱書號與實體書號。

```
SELECT L.LoanId,M.Name,C.Title
FROM LOAN AS L,MEMBER AS M,COPY AS C
WHERE L.MemberId=M.MemberId
ORDER BY L.LoanId,C.CopyId;
```

列 / 欄	B1	B2	B3	B4
L1	L1/B1	L1/B2	L1/B3	L1/B4
L2	L2/B1	L2/B2	L2/B3	L2/B4
L3	L3/B1	L3/B2	L3/B3	L3/B4
L4	L4/B1	L4/B2	L4/B3	L4/B4
L5	L5/B1	L5/B2	L5/B3	L5/B4
L6	L6/B1	L6/B2	L6/B3	L6/B4

完整24列：每筆借閱配四本實體，6×4 = 24；矩陣不是 SQL 的輸出格式。L1/B2 表示 L1、小安、SQL_練習，這不是 L1 的真正借書。

三表所有組合原有6×4×4 = 96組。會員條件讓每筆借閱只配自己的1位會員，所以剩6×1×4 = 24，不再乘四位會員。

同書名的 B1、B4 仍各形成一列；沒有 DISTINCT 時，相同輸出也不會自動合併。

本頁重點：只漏書號條件就把六筆擴成24筆；會員配對正確不表示書也配對正確。

圖書館查詢 B：補書號條件留下六列

起點：第62、63頁的完整圖書館資料，六筆借閱。與 A 輸出欄、資料相同；B 只補 L.CopyId=C.CopyId。先預測 L1 是否只留下 SQL入門。

```
SELECT L.LoanId,M.Name,C.Title
FROM LOAN AS L,MEMBER AS M,COPY AS C
WHERE L.MemberId=M.MemberId AND L.CopyId=C.CopyId
ORDER BY L.LoanId,C.CopyId;
```

LoanId	Name	Title
L1	小安	SQL入門
L2	小安	SQL入門
L3	小文	SQL_練習
L4	小安	DB設計
L5	小文	SQL入門
L6	小安	SQL入門

問題：兩位小安、兩本 SQL入門 為什麼不會亂配？

回答：WHERE 比的是 MemberId、CopyId，不比姓名或書名；結果欄顯示名稱不會改變配對依據。

本頁重點：兩條配對條件各負責一個對象；六筆借閱各得到自己的會員與實體書。

DISTINCT：六次借閱、三位借閱會員

起點：第62、63頁的完整圖書館資料，六筆借閱。要列借過書的每位會員一次，不是列每次借閱。先預測 M4 是否出現。

```
SELECT MemberId FROM LOAN ORDER BY MemberId;
```

```
SELECT DISTINCT MemberId FROM LOAN ORDER BY MemberId;
```

A：6列，會員重複

MemberId
M1
M1
M2
M2
M3
M3

B：3列，每位一次

MemberId
M1
M2
M3

問題：只用姓名去重，還能表示三位嗎？

回答：不能。M1、M3 都叫小安；保留 MemberId 才能分清三位，沒有借閱的 M4 不在本題。

本頁重點：DISTINCT 對輸出欄去重；本題輸出會員識別，不輸出借閱識別或只輸出姓名。

UNION：合併借過兩本指定實體的會員

起點：第62、63頁的完整圖書館資料，六筆借閱。B1 借閱名單是 M1、M1、M3；B2 是 M2。目標是借過其中任一本的每位會員只列一次。

```
SELECT MemberId FROM LOAN WHERE CopyId='B1'
UNION ALL
SELECT MemberId FROM LOAN WHERE CopyId='B2'
ORDER BY MemberId;
```

```
SELECT MemberId FROM LOAN WHERE CopyId='B1'
UNION
SELECT MemberId FROM LOAN WHERE CopyId='B2'
ORDER BY MemberId;
```

A：UNION ALL，4列

MemberId
M1
M1
M2
M3

B：UNION，3列

MemberId
M1
M2
M3

重複來自 M1 的兩次 B1 借閱。借過 B3 的資料不直接加入這兩份查詢。

本頁重點：相同問題，B 去除重複會員；合併的是查詢結果，不是把兩本實體合成同一本。

LIKE：底線字元與底線萬用字元

起點：第62、63頁的完整圖書館資料，六筆借閱。要查書名以字面 SQL_ 開頭的實體，也就是 B2。A 未跳脫底線；B 用 ! 指出下一字元按字面比對。

```
SELECT CopyId,Title FROM COPY
WHERE Title LIKE 'SQL_%' ORDER BY CopyId;
```

```
SELECT CopyId,Title FROM COPY
WHERE Title LIKE 'SQL!_%' ESCAPE '!' ORDER BY CopyId;
```

A：查3本，多2本

CopyId	Title
B1	SQL入門
B2	SQL_練習
B4	SQL入門

B：只查到 B2

CopyId	Title
B2	SQL_練習

A 的 _ 可以配 SQL入門 的「入」；B 的 ! 不比對書名字元，只把後面的 _ 當字面底線。

本頁重點：答案是 B2 一筆；本例在 SQLite 執行。B1、B4 同名卻各是不同實體。

BETWEEN：日期範圍含兩個端點

起點：第62、63頁的完整圖書館資料，六筆借閱。目標是9/1到9/10借出的事件，包含9/1、9/10。相同固定日期格式；先預測 L1、L2 應不應出現。

```
SELECT LoanId,BorrowedOn FROM LOAN
WHERE BorrowedOn>'2026-09-01' AND BorrowedOn<'2026-09-10'
ORDER BY BorrowedOn,LoanId;
```

```
SELECT LoanId,BorrowedOn FROM LOAN
WHERE BorrowedOn BETWEEN '2026-09-01' AND '2026-09-10'
ORDER BY BorrowedOn,LoanId;
```

A：嚴格不等式，2列

LoanId	BorrowedOn
L4	2026-09-04
L5	2026-09-06

B：包含端點，4列

LoanId	BorrowedOn
L1	2026-09-01
L4	2026-09-04
L5	2026-09-06
L2	2026-09-10

本頁重點：A 漏 L1、L2；B 包含端點。寫 >= 與 <= 也能得到 B 的結果，不是只能用 BETWEEN。

INSERT：補登借閱要用新的 LoanId

起點：第62、63頁的完整圖書館資料，六筆借閱。補登 M1 在8/10借 B1、8/15還的一次事件；它早於9月兩次借閱。不是更正原 L1，應新增獨立事件。A、B 各從六筆原資料開始。

```
INSERT INTO LOAN VALUES
  ('L1','M1','B1','2026-08-10','2026-08-15');
```

A：L1 已存在，主鍵拒絕
六筆原借閱全保留；沒有覆寫9月的 L1。

```
INSERT INTO LOAN VALUES
  ('L7','M1','B1','2026-08-10','2026-08-15');
SELECT LoanId,BorrowedOn,ReturnedOn FROM LOAN
ORDER BY BorrowedOn,LoanId;
```

LoanId	BorrowedOn	ReturnedOn
L7	2026-08-10	2026-08-15
L1	2026-09-01	2026-09-05
L4	2026-09-04	2026-09-08
L5	2026-09-06	2026-09-09
L2	2026-09-10	2026-09-15
L3	2026-09-11	NULL
L6	2026-09-16	NULL

本頁重點：B 用 L7 成功加一筆，共七筆；下一個獨立比較仍回六筆原資料，不沿用新增結果。

副本主鍵：同一筆 L3 能否重複新增

目標：每次借閱只保留一列。A、B 各另開空資料庫，依第64–68頁建表載入六筆；都尚無副本。實作額外補充（SQLite）。

A：直接複製，再重複加入 L3；預測 SELECT 會顯示幾列。

```
CREATE TABLE LOAN_SNAPSHOT AS SELECT LoanId,ReturnedOn FROM LOAN;
INSERT INTO LOAN_SNAPSHOT
SELECT LoanId,ReturnedOn FROM LOAN WHERE LoanId='L3';
SELECT LoanId FROM LOAN_SNAPSHOT ORDER BY LoanId;
```

B：另從上述六筆起點開始，先定義副本主鍵，再做相同新增。

```
CREATE TABLE LOAN_SNAPSHOT (
  LoanId TEXT NOT NULL PRIMARY KEY,ReturnedOn TEXT);
INSERT INTO LOAN_SNAPSHOT SELECT LoanId,ReturnedOn FROM LOAN;
INSERT INTO LOAN_SNAPSHOT
SELECT LoanId,ReturnedOn FROM LOAN WHERE LoanId='L3';
```

B 報 UNIQUE constraint failed: LOAN_SNAPSHOT.LoanId 後，另執行：

```
SELECT LoanId FROM LOAN_SNAPSHOT ORDER BY LoanId;
```

A：7 列，L3 重複

LoanId
L1
L2
L3
L3
L4
L5
L6

B：6 列，拒絕重複

LoanId
L1
L2
L3
L4
L5
L6

兩次 SELECT 的完整結果；AS SELECT 沒有帶入主鍵，B 的主鍵才擋下第二筆 L3。

本頁重點：複製資料與建立限制要分開確認；同一個新增在 A 通過、B 被拒絕。

保留歷史：RESTRICT 與 CASCADE 的差異

起點：第62、63頁的完整圖書館資料，六筆借閱。圖書館正式需求是保留歷史，不能因刪會員連帶丟掉借閱。A 只在另建測試版把 LOAN 的 MemberId 改 ON DELETE CASCADE；B 使用正式 RESTRICT。兩邊資料相同。

A 新無表資料庫：依第64–68頁建表載入，只有 LOAN 的 ON DELETE RESTRICT 換 CASCADE。

```
DELETE FROM MEMBER WHERE MemberId='M1';
SELECT LoanId,MemberId FROM LOAN ORDER BY LoanId;
```

B 另開新無表資料庫，依同一組頁面原樣建表載入；預測是否保留 M1、L1、L2。

```
DELETE FROM MEMBER WHERE MemberId='M1';
```

B 報錯後，另執行以下查詢；不重做建表或刪除。

```
SELECT LoanId,MemberId FROM LOAN ORDER BY LoanId;
```

A：CASCADE，剩4筆

LoanId	MemberId
L3	M2
L4	M3
L5	M2
L6	M3

B：RESTRICT，保留6筆

LoanId	MemberId
L1	M1
L2	M1
L3	M2
L4	M3
L5	M2
L6	M3

A 丟掉 L1、L2 歷史。B 回傳 FOREIGN KEY constraint failed，M1 未刪，會員四列全保留。

本頁重點：符合本故事的是 B。CASCADE 不一定錯，但不符合此處要保留會員借閱歷史的需求。

DELETE：只刪核准的誤登借閱

本頁假設已確認 L3 誤登，只准刪 L3。A、B 各另開空資料庫，依第64–68頁建表載入六筆；B 不接續 A 刪完的資料。

```
DELETE FROM LOAN;
SELECT LoanId FROM LOAN ORDER BY LoanId;
```

A：漏 WHERE，六筆全刪
LOAN 剩0列；會員與實體書沒有因此自動刪掉。

```
DELETE FROM LOAN WHERE LoanId='L3';
SELECT LoanId FROM LOAN ORDER BY LoanId;
```

LoanId
L1
L2
L4
L5
L6

B 剩 L1、L2、L4、L5、L6，會員四列與實體四列都保留。

本頁重點：WHERE LoanId 指定一次事件；刪會員或實體書則要檢查是否仍有借閱參照。

UPDATE：只登記 L3 的歸還日期

只登錄 L3 在9/16還書，L6 仍未還。A、B 各另開空資料庫，依第64–68頁建表載入六筆；B 不接續 A 改過的日期。先預測漏 WHERE 會不會被 CHECK 擋住。

```
UPDATE LOAN SET ReturnedOn='2026-09-16';
SELECT LoanId,ReturnedOn FROM LOAN ORDER BY LoanId;
```

```
UPDATE LOAN SET ReturnedOn='2026-09-16' WHERE LoanId='L3';
SELECT LoanId,ReturnedOn FROM LOAN ORDER BY LoanId;
```

A：六筆都改成9/16

LoanId	ReturnedOn
L1	2026-09-16
L2	2026-09-16
L3	2026-09-16
L4	2026-09-16
L5	2026-09-16
L6	2026-09-16

B：只改 L3

LoanId	ReturnedOn
L1	2026-09-05
L2	2026-09-15
L3	2026-09-16
L4	2026-09-08
L5	2026-09-09
L6	NULL

本頁重點：A 的日期都未早於借出日，所以 CHECK 不會擋；但範圍全錯。B 用 WHERE 才只改已核對的 L3。